Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №8

“ Мережеве та асинхронне програмування. REST API ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON.

Варіант 10. Система обліку складських запасів

Ресурси: products, stock_operations.

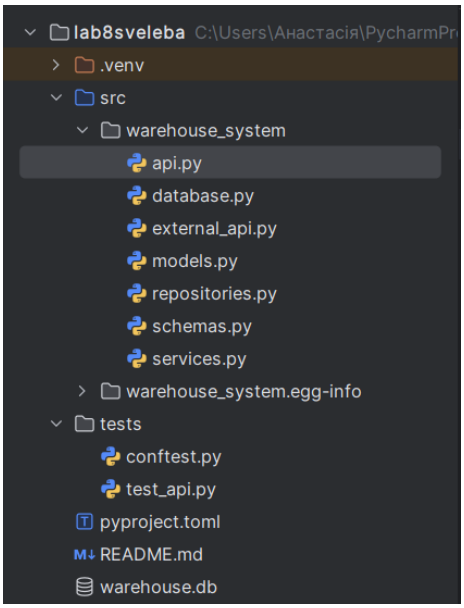
Endpoints: GET /products; GET /products/{id}; POST /products; PATCH /products/{id}; POST /products/{id}/incoming; POST /products/{id}/write-off; GET /products/low-stock; GET /stock/operations; GET /stock/value.

Реалізувати: додавання товару; надходження; списання; пошук; зміну ціни; загальну вартість складу; низький запас; історію руху. Не дозволяти списання понад залишок.

Асинхронне завдання: паралельно перевіряти залишки декількох товарів; Task/gather, Semaphore, HTTPX, timeout, retry.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse_system/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/.



2. У файлі pyproject.toml ви налаштували метадані пакета warehouse-system, вказали сумісність із Python версії 3.11 та вище, а також визначили необхідні залежності (fastapi, uvicorn, sqlalchemy, pydantic, httpx) та опціональні залежності для тестування (pytest, pytest-asyncio).

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
```

```
name = "warehouse-system"
version = "0.8.0"
description = "Warehouse Management System REST API (Variant 10)"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "fastapi>=0.110.0",
    "uvicorn>=0.28.0",
    "sqlalchemy>=2.0.0",
    "pydantic>=2.6.0",
    "httpx>=0.27.0",
]

[project.optional-dependencies]
test = [
    "pytest>=8.0.0",
    "pytest-asyncio>=0.23.0",
]

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

3. У файлі README.md ви описали призначення проєкту, загальну структуру компонентів, а також додали інструкції зі встановлення залежностей, запуску сервера розробки через Uvicorn, перегляду документації в Swagger та виконання автоматизованих тестів.

```
# Warehouse Management System (REST API) - Variant 10

Система обліку складських запасів із підтримкою асинхронного REST API, створена за допомогою FastAPI, Pydantic, SQLAlchemy та HTTPX.

## Структура проєкту

- `src/warehouse_system/database.py` — налаштування з'єднання з БД та SQLAlchemy session.
- `src/warehouse_system/models.py` — SQLAlchemy ORM-моделі `Product` та `StockOperation`.
- `src/warehouse_system/schemas.py` — Pydantic-схеми для валідації та серіалізації даних.
- `src/warehouse_system/repositories.py` — Repository Layer для ізольованої роботи з БД.
- `src/warehouse_system/services.py` — Service Layer для бізнес-логіки та асинхронних операцій.
- `src/warehouse_system/api.py` — FastAPI застосунок та REST endpoints.
- `src/warehouse_system/external_api.py` — асинхронні HTTP-запити через HTTPX (gather, Task, Semaphore, retry, timeout).
- `tests/` — автоматизовані тести API та бізнес-логіки.

## Інструкція з запуску

1. Встановлення залежностей:
```bash
pip install -e .[test]
```
Запуск сервера розробки:
```bash
uvicorn warehouse_system.api:app --reload
```

Інтерактивна документація Swagger:
Відкрийте у браузері: http://127.0.0.1:8000/docs
```

```
## Запуск автоматизованих тестів:
```bash
pytest -v
```
```

4. У файлі src/warehouse_system/database.py ви налаштували підключення до бази даних SQLite (warehouse.db), створили фабрику сесій SessionLocal та реалізували залежність get_session для отримання сесії бази даних у FastAPI ендпоінтах.

```
from collections.abc import Generator
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

DATABASE_URL = "sqlite:///./warehouse.db"

engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


def get_session() -> Generator[Session, None, None]:
    """Dependency for obtaining a database session."""
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()
```

5. У файлі src/warehouse_system/models.py ви визначили декларативну базу SQLAlchemy та описали ORM-моделі Product (товар із назвою, артикулом, ціною, кількістю та мін. рівнем запасу) та StockOperation (операції надходження й списання із зовнішнім ключем та каскадним видаленням).

```
from sqlalchemy import Column, Float, ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, relationship


class Base(DeclarativeBase):
    pass


class Product(Base):
    __tablename__ = "products"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    sku = Column(String, unique=True, index=True, nullable=False)
    price = Column(Float, nullable=False)
    quantity = Column(Integer, default=0, nullable=False)
    min stock level = Column(Integer, default=5, nullable=False)

    operations = relationship(
        "StockOperation", back_populates="product", cascade="all, delete-orphan"
    )


class StockOperation(Base):
    __tablename__ = "stock_operations"
```

```
    id = Column(Integer, primary_key=True, index=True)
    product_id = Column(
        Integer, ForeignKey("products.id", ondelete="CASCADE"), nullable=False
    )
    operation_type = Column(
        String, nullable=False
    )  # 'incoming' або 'write-off'
    amount = Column(Integer, nullable=False)
    comment = Column(String, nullable=True)

    product = relationship("Product", back_populates="operations")
```

6. У файлі src/warehouse_system/schemas.py ви створили Pydantic-моделі (ProductCreate, ProductUpdate, ProductResponse, StockOperationCreate, StockOperationResponse, WarehouseValueResponse) для валідації вхідних JSON-даних та серіалізації відповідей API.

```
from pydantic import BaseModel, Field


class ProductCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    sku: str = Field(min_length=2, max_length=50)
    price: float = Field(gt=0)
    quantity: int = Field(ge=0)
    min_stock_level: int = Field(ge=0, default=5)


class ProductUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=100)
    sku: str | None = Field(default=None, min_length=2, max_length=50)
    price: float | None = Field(default=None, gt=0)
    min_stock_level: int | None = Field(default=None, ge=0)


class ProductResponse(BaseModel):
    id: int
    name: str
    sku: str
    price: float
    quantity: int
    min_stock_level: int

    model_config = {"from_attributes": True}


class StockOperationCreate(BaseModel):
    amount: int = Field(gt=0)
    comment: str | None = None


class StockOperationResponse(BaseModel):
    id: int
    product_id: int
    operation_type: str
    amount: int
    comment: str | None

    model_config = {"from_attributes": True}


class WarehouseValueResponse(BaseModel):
```

```
    total_stock_value: float
    total_products_count: int
```

7. У файлі src/warehouse_system/repositories.py ви реалізували класи ProductRepository та StockOperationRepository для ізоляції логіки роботи з базою даних, виконання SQL-запитів (select, get, add, delete) та отримання товарів із низьким запасом.

```
from sqlalchemy import select
from sqlalchemy.orm import Session
from warehouse_system.models import Product, StockOperation


class ProductRepository:

    def __init__(self, session: Session) -> None:
        self.session = session

    def get_all(self) -> list[Product]:
        statement = select(Product).order_by(Product.name)
        return list(self.session.scalars(statement))

    def get_by_id(self, product_id: int) -> Product | None:
        return self.session.get(Product, product_id)

    def get_by_sku(self, sku: str) -> Product | None:
        statement = select(Product).where(Product.sku == sku)
        return self.session.scalar(statement)

    def add(self, product: Product) -> Product:
        self.session.add(product)
        self.session.commit()
        self.session.refresh(product)
        return product

    def delete(self, product: Product) -> None:
        self.session.delete(product)
        self.session.commit()

    def get_low_stock(self) -> list[Product]:
        statement = select(Product).where(
            Product.quantity <= Product.min_stock_level
        )
        return list(self.session.scalars(statement))


class StockOperationRepository:

    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, operation: StockOperation) -> StockOperation:
        self.session.add(operation)
        self.session.commit()
        self.session.refresh(operation)
        return operation

    def get_all(self) -> list[StockOperation]:
        statement = select(StockOperation).order_by(StockOperation.id.desc())
        return list(self.session.scalars(statement))
```

8. У файлі src/warehouse_system/services.py ви реалізували бізнес-логіку застосунку в класі WarehouseService, зокрема перевірку на унікальність артикулу, оновлення/видалення товарів, а також ключове бізнес-правило Варіанта 10 — заборону списання товарів понад наявний залишок.

```
from sqlalchemy.orm import Session
from warehouse_system.models import Product, StockOperation
from warehouse_system.repositories import (
    ProductRepository,
    StockOperationRepository,
)
from warehouse_system.schemas import (
    ProductCreate,
    ProductUpdate,
    StockOperationCreate,
)


class WarehouseService:

    def __init__(self, session: Session) -> None:
        self.session = session
        self.product_repo = ProductRepository(session)
        self.operation_repo = StockOperationRepository(session)

    def list_products(self) -> list[Product]:
        return self.product_repo.get_all()

    def get_product(self, product_id: int) -> Product | None:
        return self.product_repo.get_by_id(product_id)

    def create_product(self, data: ProductCreate) -> Product:
        existing = self.product_repo.get_by_sku(data.sku)
        if existing:
            raise ValueError(
                f"Product with SKU '{data.sku}' already exists."
            )

        product = Product(
            name=data.name,
            sku=data.sku,
            price=data.price,
            quantity=data.quantity,
            min_stock_level=data.min_stock_level,
        )
        return self.product_repo.add(product)

    def update_product(
        self, product_id: int, data: ProductUpdate
    ) -> Product | None:
        product = self.product_repo.get_by_id(product_id)
        if not product:
            return None

        updates = data.model_dump(exclude_unset=True)
        for field, value in updates.items():
            setattr(product, field, value)

        self.session.commit()
        self.session.refresh(product)
        return product

    def delete_product(self, product_id: int) -> bool:
        product = self.product_repo.get_by_id(product_id)
```

```
        if not product:
            return False
        self.product_repo.delete(product)
        return True

    def process_incoming(
        self, product_id: int, data: StockOperationCreate
    ) -> StockOperation:
        product = self.product_repo.get_by_id(product_id)
        if not product:
            raise KeyError("Product not found")

        product.quantity += data.amount
        operation = StockOperation(
            product_id=product.id,
            operation_type="incoming",
            amount=data.amount,
            comment=data.comment,
        )
        return self.operation_repo.add(operation)

    def process_write_off(
        self, product_id: int, data: StockOperationCreate
    ) -> StockOperation:
        product = self.product_repo.get_by_id(product_id)
        if not product:
            raise KeyError("Product not found")

        # Перевірка умови з Варіанта 10: не дозволяти списання понад залишок
        if product.quantity < data.amount:
            raise ValueError("Write-off amount exceeds available stock.")

        product.quantity -= data.amount
        operation = StockOperation(
            product_id=product.id,
            operation_type="write-off",
            amount=data.amount,
            comment=data.comment,
        )
        return self.operation_repo.add(operation)

    def get_low_stock(self) -> list[Product]:
        return self.product_repo.get_low_stock()

    def get_operations(self) -> list[StockOperation]:
        return self.operation_repo.get_all()

    def get_total_value(self) -> dict[str, float | int]:
        products = self.product_repo.get_all()
        total_val = sum(p.price * p.quantity for p in products)
        return {
            "total_stock_value": round(total_val, 2),
            "total_products_count": len(products),
        }
```

9. У файлі src/warehouse_system/external_api.py ви реалізували асинхронні взаємодії через клієнт HTTPX, налаштували таймаути, логіку повторних спроб (retry), паралельне виконання завдань за допомогою asyncio.Task і asyncio.gather(), а також обмеження кількості одночасних операцій через asyncio.Semaphore.

```
import asyncio
import httpx
```

```python
async def fetch_with_retry(
    client: httpx.AsyncClient, url: str, max_attempts: int = 3
) -> dict:
    """Виконує HTTP GET запит із можливістю повторних спроб (retry)."""
    for attempt in range(1, max_attempts + 1):
        try:
            response = await client.get(url)
            response.raise_for_status()
            return response.json()
        except (httpx.RequestError, httpx.HTTPStatusError):
            if attempt == max_attempts:
                raise
            await asyncio.sleep(attempt)
    raise RuntimeError("Unexpected retry state")


async def check_multiple_products_stock(
    base_url: str, product_ids: list[int]
) -> list[dict]:
    """Паралельно отримує дані про залишки товарів із використанням Semaphore та
Task/gather."""
    semaphore = asyncio.Semaphore(3)  # Обмеження паралелізму

    async def limited_fetch(client: httpx.AsyncClient, pid: int) -> dict:
        url = f"{base_url}/products/{pid}"
        async with semaphore:
            return await fetch_with_retry(client, url)

    async with httpx.AsyncClient(timeout=5.0) as client:
        # Створення списку asyncio.Task
        tasks = [
            asyncio.create_task(limited_fetch(client, pid))
            for pid in product_ids
        ]

        try:
            # Паралельне виконання з таймаутом
            results = await asyncio.wait_for(
                asyncio.gather(*tasks), timeout=10.0
            )
            return list(results)
        except asyncio.TimeoutError:
            return []
```

10. У файлі src/warehouse_system/api.py ви створили FastAPI-застосунок, налаштували автоматичне створення таблиць БД та реалізували REST endpoints для управління товарами (GET, POST, PATCH, DELETE), складськими операціями (надходження, списання, історія, загальна вартість) та пакетною перевіркою залишків.

```python
from fastapi import Depends, FastAPI, HTTPException, status
from sqlalchemy.orm import Session
from warehouse_system.database import engine, get_session
from warehouse_system.external_api import check_multiple_products_stock
from warehouse_system.models import (  # <--- Імпортуємо Base з models, а не з
database
    Base,
    Product,
    StockOperation,
)
from warehouse_system.schemas import (
```

```
    ProductCreate,
    ProductResponse,
    ProductUpdate,
    StockOperationCreate,
    StockOperationResponse,
    WarehouseValueResponse,
)
from warehouse_system.services import WarehouseService

# Створення таблиць БД при старті
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Warehouse Management System API",
    version="1.0.0",
    description="REST API для системи обліку складських запасів (Варіант 10)",
)
from fastapi import Depends, FastAPI, HTTPException, status
from sqlalchemy.orm import Session
from warehouse_system.database import get_session
from warehouse_system.external_api import check_multiple_products_stock
from warehouse_system.models import (
    Product,
    StockOperation,
)
from warehouse_system.schemas import (
    ProductCreate,
    ProductResponse,
    ProductUpdate,
    StockOperationCreate,
    StockOperationResponse,
    WarehouseValueResponse,
)
from warehouse_system.services import WarehouseService

app = FastAPI(
    title="Warehouse Management System API",
    version="1.0.0",
    description="REST API для системи обліку складських запасів (Варіант 10)",
)


@app.get("/products", response_model=list[ProductResponse])
async def get_products(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.list_products()


@app.get("/products/low-stock", response_model=list[ProductResponse])
async def get_low_stock_products(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_low_stock()


@app.get("/products/{product_id}", response_model=ProductResponse)
async def get_product(product_id: int, session: Session = Depends(get_session)):
    service = WarehouseService(session)
    product = service.get_product(product_id)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    return product


@app.post(
```

```
    "/products",
    response_model=ProductResponse,
    status_code=status.HTTP_201_CREATED,
)
async def create_product(
    data: ProductCreate, session: Session = Depends(get_session)
):
    service = WarehouseService(session)
    try:
        return service.create_product(data)
    except ValueError as err:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=str(err)
        )


@app.patch("/products/{product_id}", response_model=ProductResponse)
async def update_product(
    product_id: int,
    data: ProductUpdate,
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    updated = service.update_product(product_id, data)
    if not updated:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    return updated


@app.delete("/products/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_product(
    product_id: int, session: Session = Depends(get_session)
):
    service = WarehouseService(session)
    success = service.delete_product(product_id)
    if not success:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )


@app.post(
    "/products/{product_id}/incoming", response_model=StockOperationResponse
)
async def product_incoming(
    product_id: int,
    data: StockOperationCreate,
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    try:
        return service.process_incoming(product_id, data)
    except KeyError:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )


@app.post(
    "/products/{product_id}/write-off", response_model=StockOperationResponse
)
async def product_write_off(
    product_id: int,
    data: StockOperationCreate,
```

```
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    try:
        return service.process_write_off(product_id, data)
    except KeyError:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    except ValueError as err:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=str(err)
        )


@app.get("/stock/operations", response_model=list[StockOperationResponse])
async def get_stock_operations(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_operations()


@app.get("/stock/value", response_model=WarehouseValueResponse)
async def get_warehouse_value(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_total_value()


@app.post("/stock/batch-check")
async def batch_check_stock(product_ids: list[int]):
    """Асинхронний ендпоінт для пакетної перевірки залишків."""
    results = await check_multiple_products_stock(
        "http://127.0.0.1:8000", product_ids
    )
    return {"results": results}

@app.get("/products", response_model=list[ProductResponse])
async def get_products(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.list_products()


@app.get("/products/low-stock", response_model=list[ProductResponse])
async def get_low_stock_products(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_low_stock()


@app.get("/products/{product_id}", response_model=ProductResponse)
async def get_product(product_id: int, session: Session = Depends(get_session)):
    service = WarehouseService(session)
    product = service.get_product(product_id)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    return product


@app.post(
    "/products",
    response_model=ProductResponse,
    status_code=status.HTTP_201_CREATED,
)
async def create_product(
    data: ProductCreate, session: Session = Depends(get_session)
):
```

```
    service = WarehouseService(session)
    try:
        return service.create_product(data)
    except ValueError as err:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=str(err)
        )


@app.patch("/products/{product_id}", response_model=ProductResponse)
async def update_product(
    product_id: int,
    data: ProductUpdate,
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    updated = service.update_product(product_id, data)
    if not updated:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    return updated


@app.delete("/products/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_product(
    product_id: int, session: Session = Depends(get_session)
):
    service = WarehouseService(session)
    success = service.delete_product(product_id)
    if not success:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )


@app.post(
    "/products/{product_id}/incoming", response_model=StockOperationResponse
)
async def product_incoming(
    product_id: int,
    data: StockOperationCreate,
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    try:
        return service.process_incoming(product_id, data)
    except KeyError:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )


@app.post(
    "/products/{product_id}/write-off", response_model=StockOperationResponse
)
async def product_write_off(
    product_id: int,
    data: StockOperationCreate,
    session: Session = Depends(get_session),
):
    service = WarehouseService(session)
    try:
        return service.process_write_off(product_id, data)
    except KeyError:
        raise HTTPException(
```

```
            status_code=status.HTTP_404_NOT_FOUND, detail="Product not found"
        )
    except ValueError as err:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=str(err)
        )


@app.get("/stock/operations", response_model=list[StockOperationResponse])
async def get_stock_operations(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_operations()


@app.get("/stock/value", response_model=WarehouseValueResponse)
async def get_warehouse_value(session: Session = Depends(get_session)):
    service = WarehouseService(session)
    return service.get_total_value()


@app.post("/stock/batch-check")
async def batch_check_stock(product_ids: list[int]):
    """Асинхронний ендпоінт для пакетної перевірки залишків."""
    results = await check_multiple_products_stock(
        "http://127.0.0.1:8000", product_ids
    )
    return {"results": results}
```

11. У файлі tests/conftest.py ви налаштували фікстуру для створення ізольованої тестової бази даних в пам'яті (sqlite:///:memory:) за допомогою SQLAlchemy для запуску тестів у середовищі pytest.

```
from collections.abc import Generator
import pytest
from sqlalchemy import create_engine
from sqlalchemy.pool import StaticPool
from sqlalchemy.orm import Session, sessionmaker
from warehouse_system.models import Base
import warehouse_system.database as db_module

TEST_DATABASE_URL = "sqlite:///:memory:"

test_engine = create_engine(
    TEST_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
TestingSessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=test_engine
)


@pytest.fixture(autouse=True)
def override_db():
    # Підміняємо engine та SessionLocal у самому модулі database на час тестів
    original_engine = db_module.engine
    original_session_local = db_module.SessionLocal

    db_module.engine = test_engine
    db_module.SessionLocal = TestingSessionLocal

    Base.metadata.create_all(bind=test_engine)
    yield
    Base.metadata.drop_all(bind=test_engine)
```

```
    db_module.engine = original_engine
    db_module.SessionLocal = original_session_local


@pytest.fixture
def db_session() -> Generator[Session, None, None]:
    session = TestingSessionLocal()
    try:
        yield session
    finally:
        session.close()
```

12. У файлі tests/test_api.py ви розробили автоматизовані інтеграційні тести API, за допомогою яких перевірили коректність створення товарів, роботу валідації Pydantic та спрацювання бізнес-обмежень (наприклад, помилку при спробі списати більше товарів, ніж є на складі).

```
from fastapi.testclient import TestClient
import pytest
from warehouse_system.api import app
from warehouse_system.database import get_session


@pytest.fixture
def client(db_session):
    def override_get_session():
        try:
            yield db_session
        finally:
            pass

    app.dependency_overrides[get_session] = override_get_session
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()


def test_create_and_get_product(client):
    response = client.post(
        "/products",
        json={
            "name": "Keyboard",
            "sku": "KB-100",
            "price": 45.0,
            "quantity": 10,
            "min_stock_level": 3,
        },
    )
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == "Keyboard"
    product_id = data["id"]

    get_res = client.get(f"/products/{product_id}")
    assert get_res.status_code == 200
    assert get_res.json()["sku"] == "KB-100"


def test_write_off_exceeds_stock(client):
    create_res = client.post(
        "/products",
        json={
            "name": "Monitor",
```

```
            "sku": "MN-500",
            "price": 200.0,
            "quantity": 2,
            "min_stock_level": 1,
        },
    )
    product_id = create_res.json()["id"]

    write_off_res = client.post(
        f"/products/{product_id}/write-off", json={"amount": 5}
    )
    assert write_off_res.status_code == 400
    assert "exceeds" in write_off_res.json()["detail"].lower()


def test_validation_error(client):
    response = client.post(
        "/products",
        json={
            "name": "X",
            "sku": "ERR",
            "price": -10.0,
            "quantity": -1,
        },
    )
    assert response.status_code == 422
```

Результати:

- Активувала віртуальне середовище, встановила залежності

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab8sveleba> python -m venv .venv
Unable to copy 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qbz5n2kfra8p0\\
сія\\PycharmProjects\\lab8sveleba\\.venv\\Scripts\\python.exe'
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab8sveleba> .venv\Scripts\activate
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab8sveleba> pip install -e .[test]
Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab8sveleba
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting fastapi>=0.110.0 (from warehouse-system==0.8.0)
  Downloading fastapi-0.159.1-py3-none-any.whl.metadata (26 kB)
Collecting uvicorn>=0.28.0 (from warehouse-system==0.8.0)
  Downloading uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Collecting sqlalchemy>=2.0.0 (from warehouse-system==0.8.0)
  Downloading sqlalchemy-2.1.6-cp313-cp313-win_amd64.whl.metadata (9.9 kB)
Collecting pydantic>=2.6.0 (from warehouse-system==0.8.0)
  Downloading pydantic-2.14.1-py3-none-any.whl.metadata (133 kB)
Collecting httpx>=0.27.0 (from warehouse-system==0.8.0)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pytest>=8.0.0 (from warehouse-system==0.8.0)
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting pytest-asyncio>=0.23.0 (from warehouse-system==0.8.0)
  Using cached pytest_asyncio-1.4.0-py3-none-any.whl.metadata (4.1 kB)
Collecting starlette>=0.46.0 (from fastapi>=0.110.0->warehouse-system==0.8.0)
  Downloading starlette-1.9.0-py3-none-any.whl.metadata (6.6 kB)
Collecting typing-extensions>=4.8.0 (from fastapi>=0.110.0->warehouse-system==0.8.0)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from fastapi>=0.110.0->warehouse-system==0.8.0)
  Building editable for warehouse-system (pyproject.toml) ... done
  Created wheel for warehouse-system: filename=warehouse_system-0.8.0-0.editable-py3-none-any.whl size=
  Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-i6bll3jc\wheels\c5\f
Successfully built warehouse-system
Installing collected packages: typing-extensions, pygments, pluggy, packaging, iniconfig, idna, h11, co
ction, sqlalchemy, pytest, pydantic-core, opentelemetry-api, httpcore, anyio, starlette, pytest-asyncio
Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 certifi-2026.10.19 click-
dna-3.20 iniconfig-2.3.1 opentelemetry-api-1.45.1 packaging-26.3 pluggy-1.7.0 pydantic-2.14.1 pydantic-
6 starlette-1.9.0 typing-extensions-4.16.0 typing-inspection-0.4.4 uvicorn-0.54.0 warehouse-system-0.8.

[notice] A new release of pip is available: 26.1.2 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Пройшла тести

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab8sveleba> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab8sveleba\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab8sveleba
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 3 items

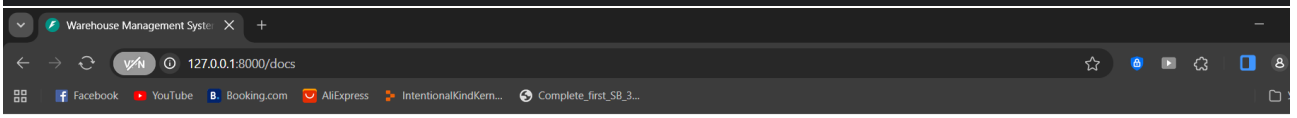
tests/test_api.py::test_create_and_get_product PASSED                    [ 33%]
tests/test_api.py::test_write_off_exceeds_stock PASSED                   [ 66%]
tests/test_api.py::test_validation_error PASSED                          [100%]

============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\Анастасія\PycharmProjects\lab8sveleba\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `h
ed; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 3 passed, 1 warning in 0.59s =========================
```

- Запустила warehouse_system.api

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab8sveleba> uvicorn warehouse_system.api:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\Анастасія\\PycharmProjects\\lab8sveleba']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [24756] using StatReload
INFO:     Started server process [21820]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



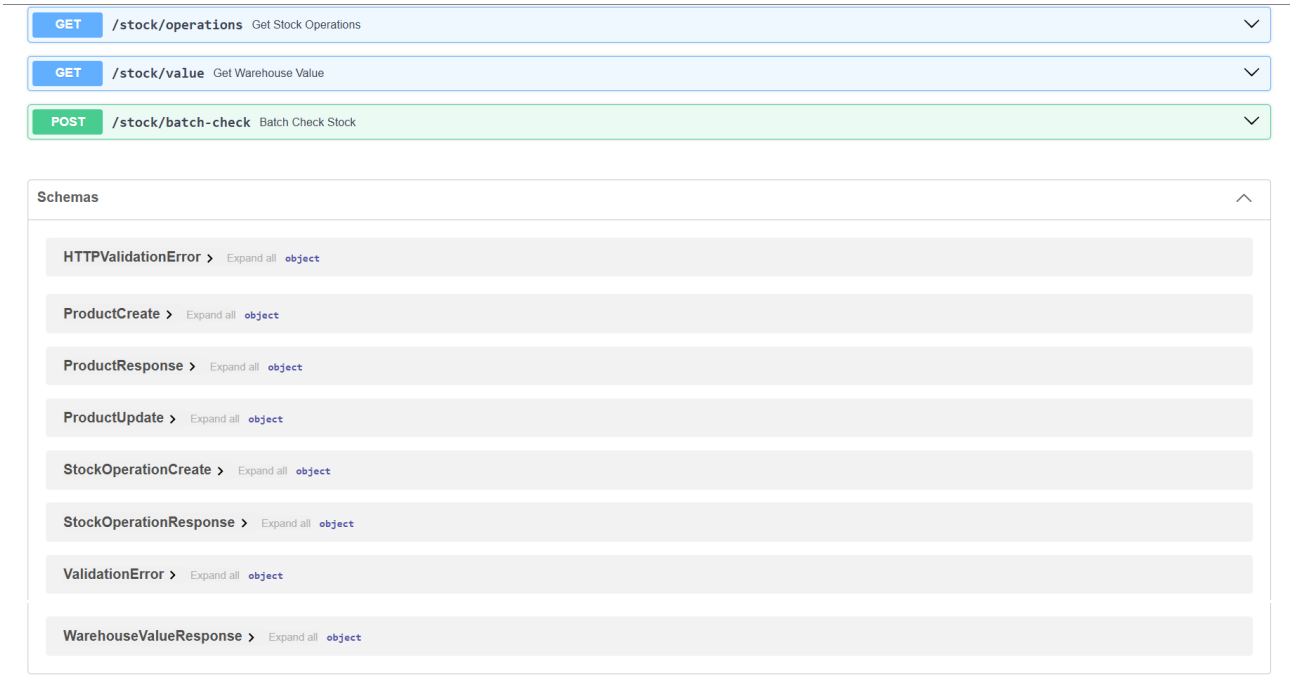
Warehouse Management System API 1.0.0 OAS 3.1

/openapi.json

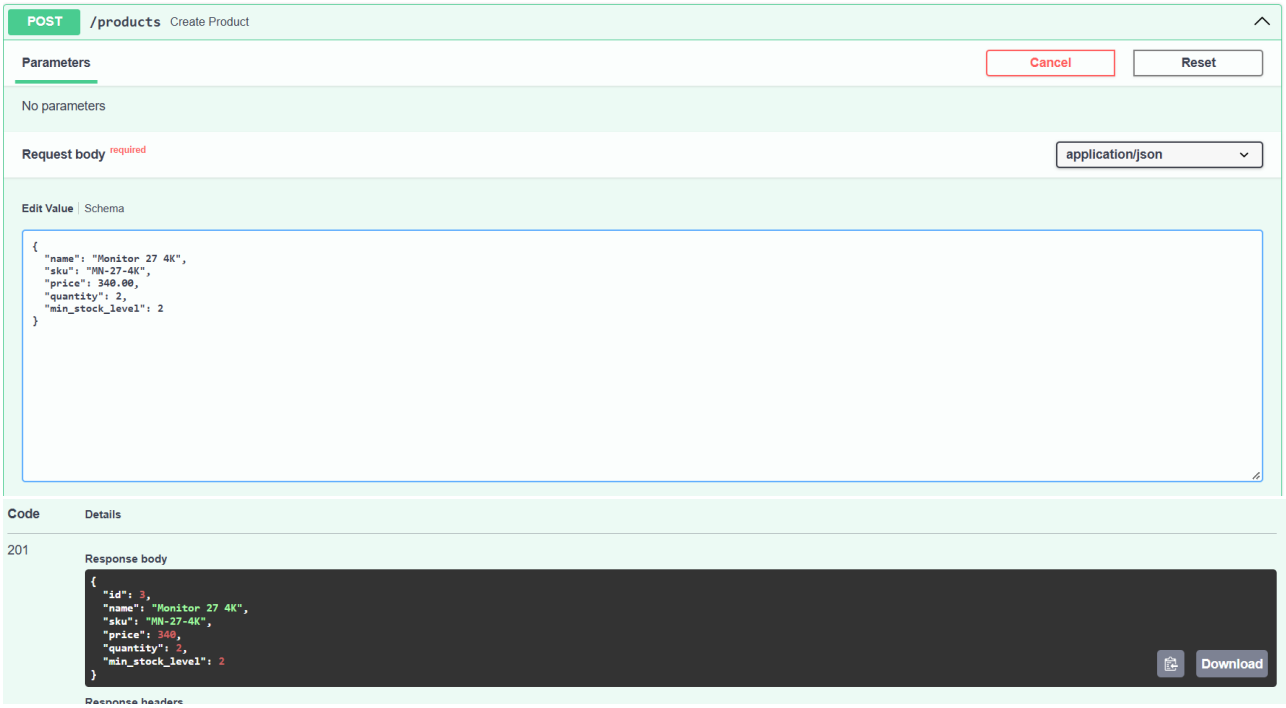
REST API для системи обліку складських запасів (Варіант 10)

default

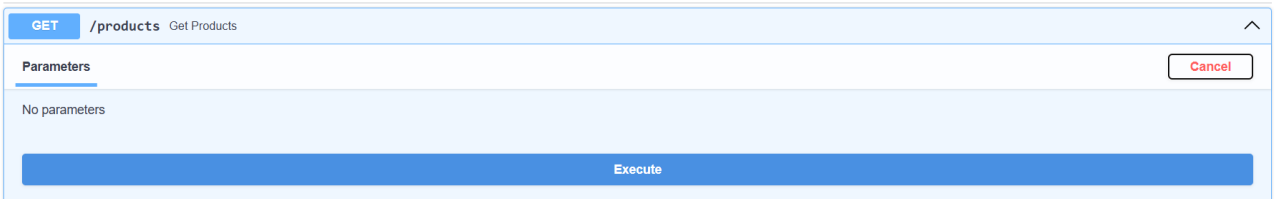
GET	/products	Get Products
POST	/products	Create Product
GET	/products/low-stock	Get Low Stock Products
GET	/products/{product_id}	Get Product
PATCH	/products/{product_id}	Update Product
DELETE	/products/{product_id}	Delete Product
POST	/products/{product_id}/incoming	Product Incoming
POST	/products/{product_id}/write-off	Product Write Off

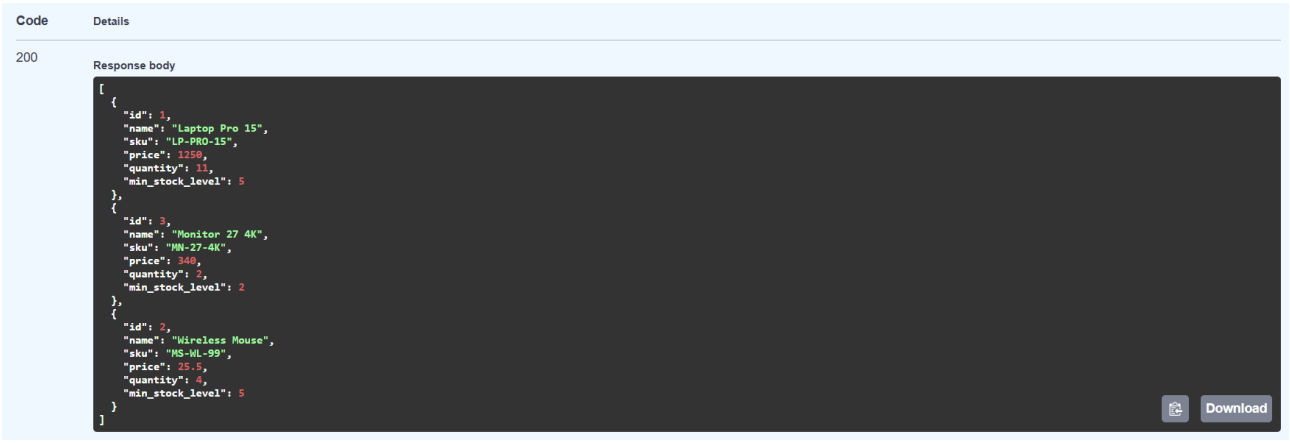


- Створила продукт

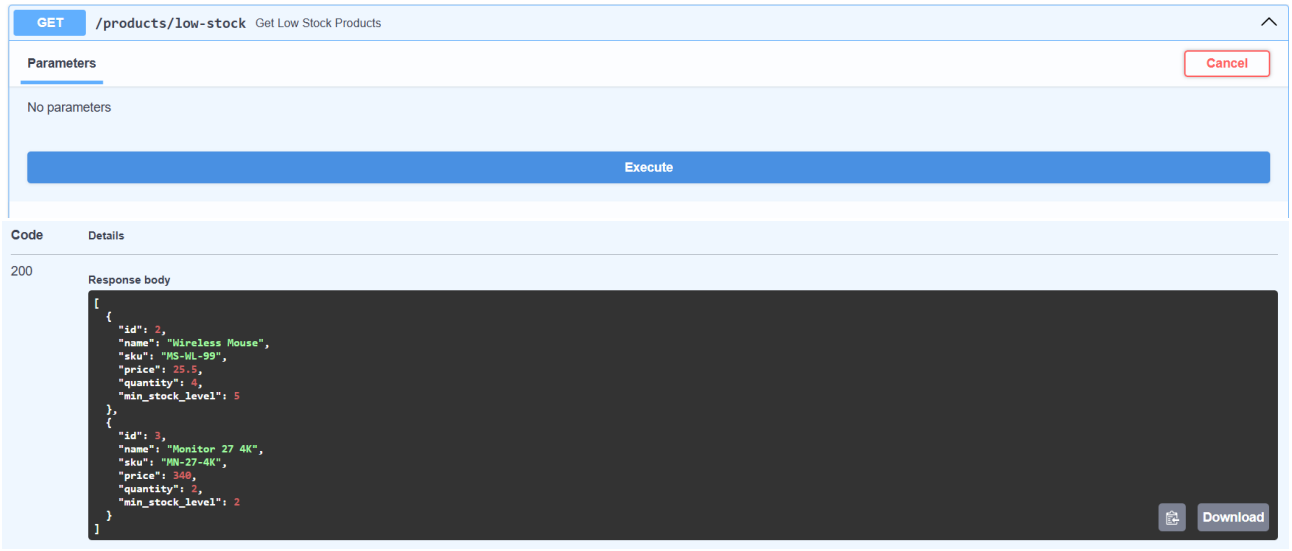


- Переглянула всі продукти

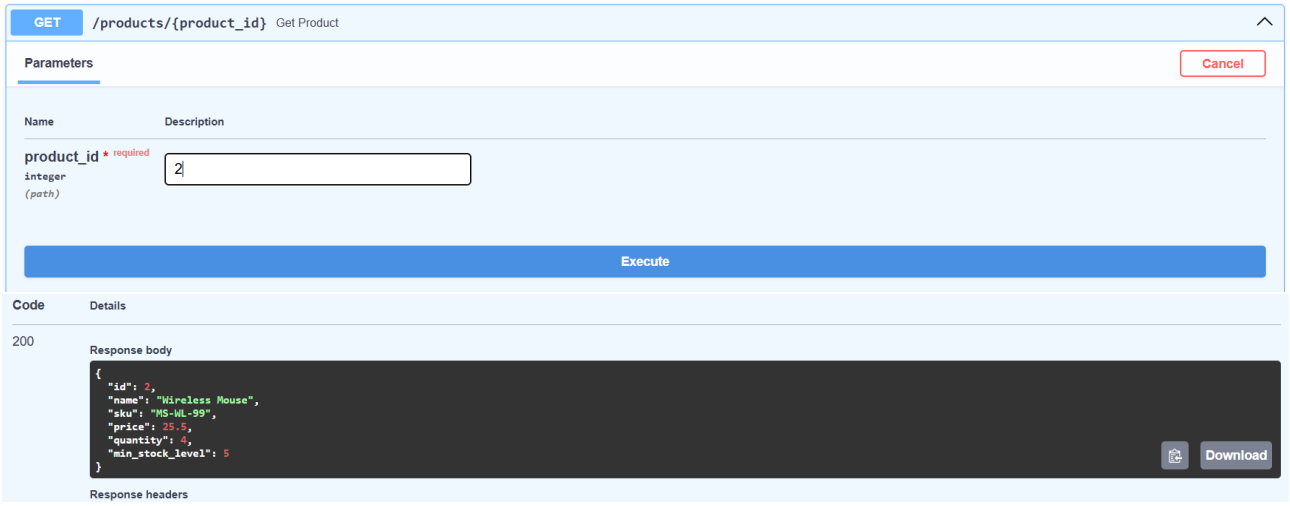




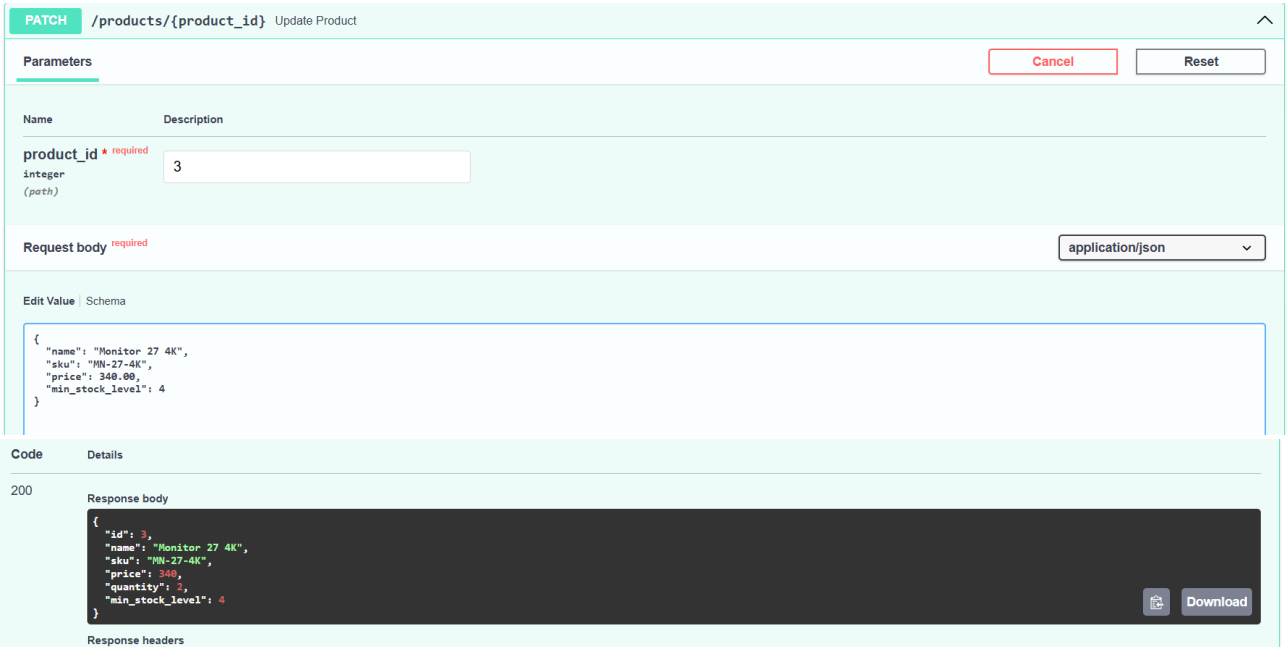
- Переглянула продукти з низьким запасом



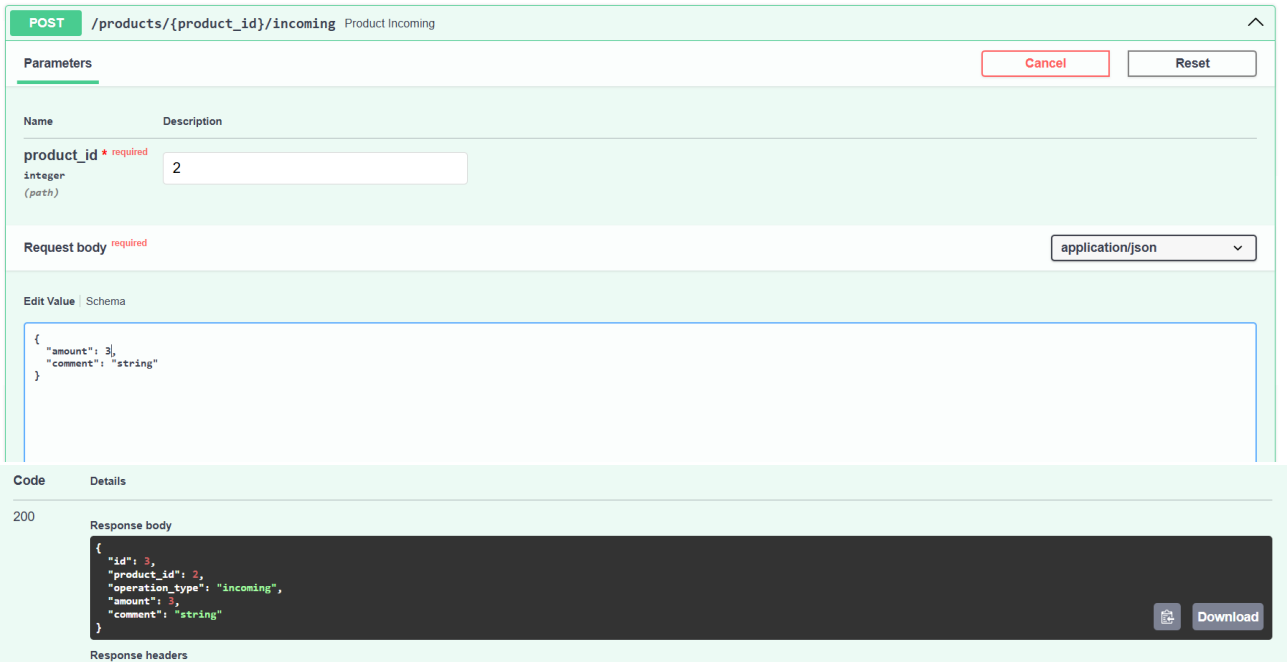
- Переглянула продукт по id



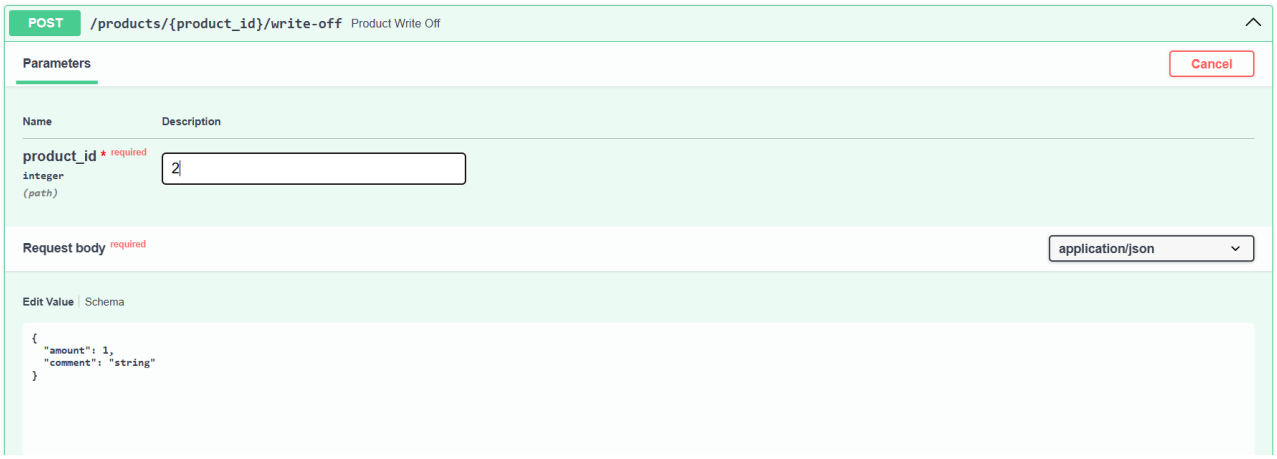
- Оновила продукт



- Ввела надходження продукту

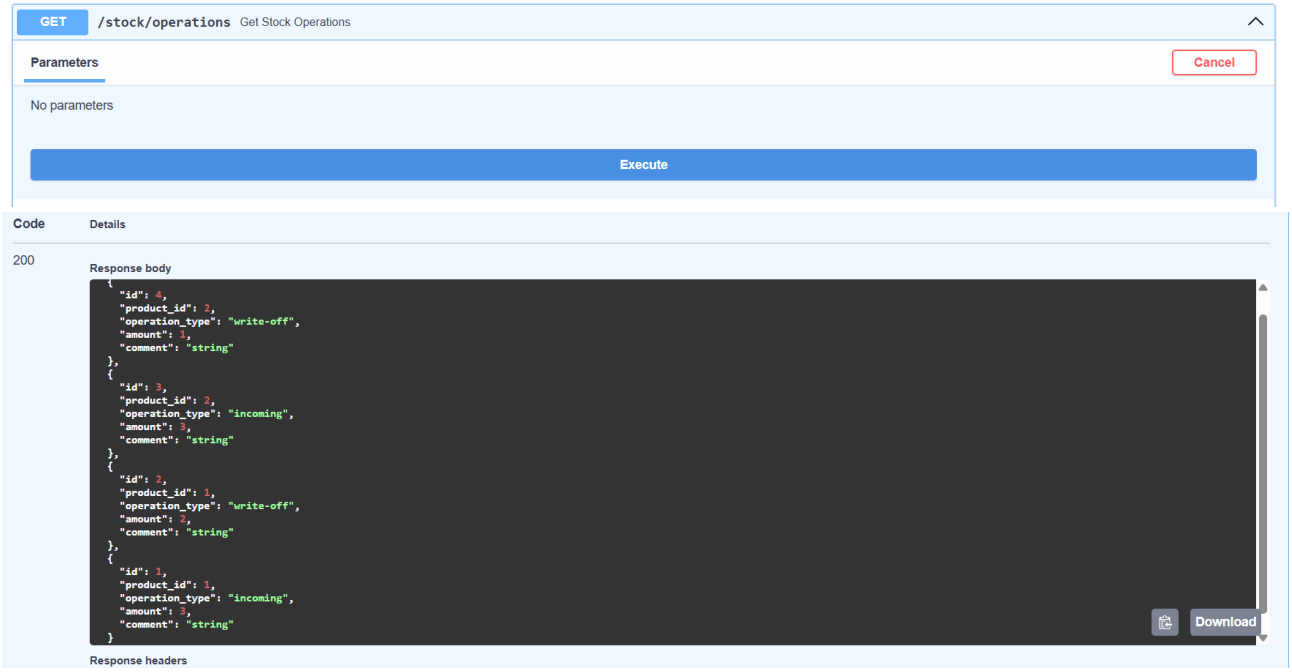


- Ввела списання продукту

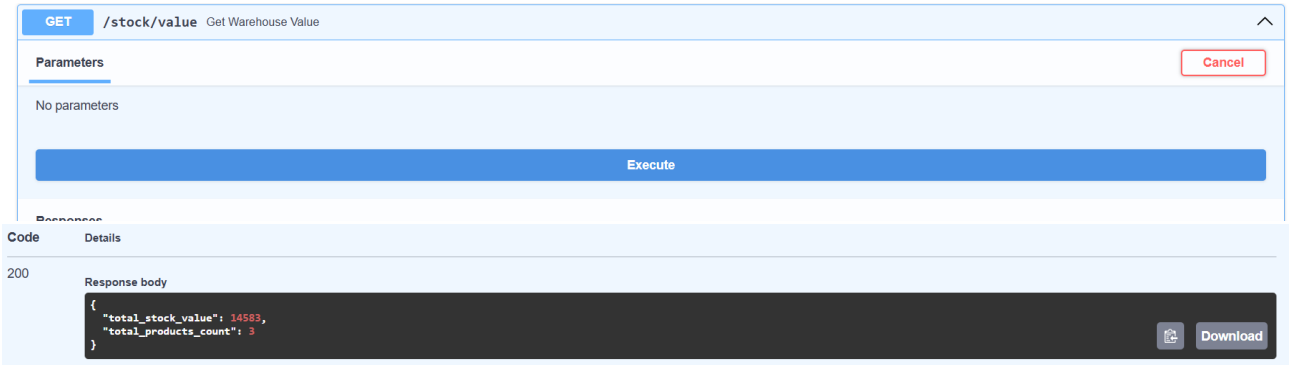




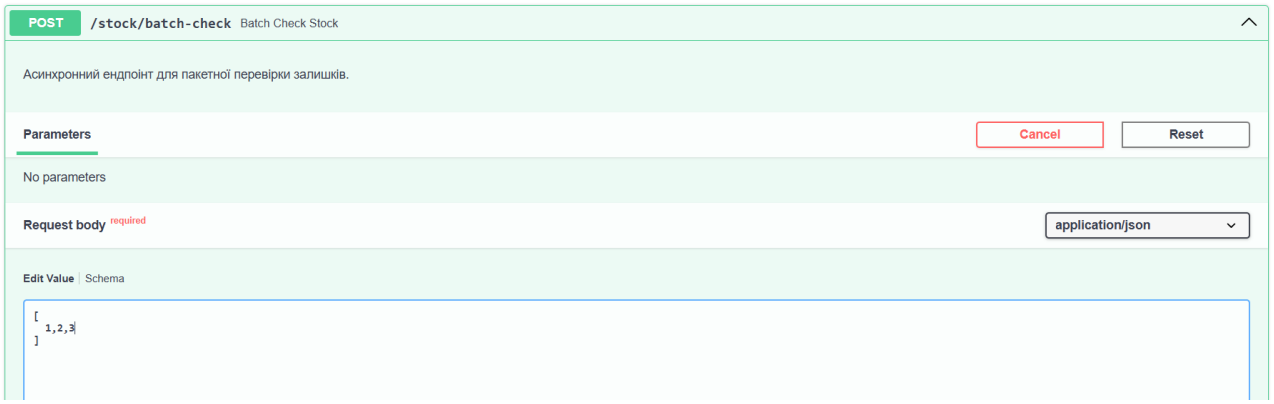
- Переглянула всі операції складу

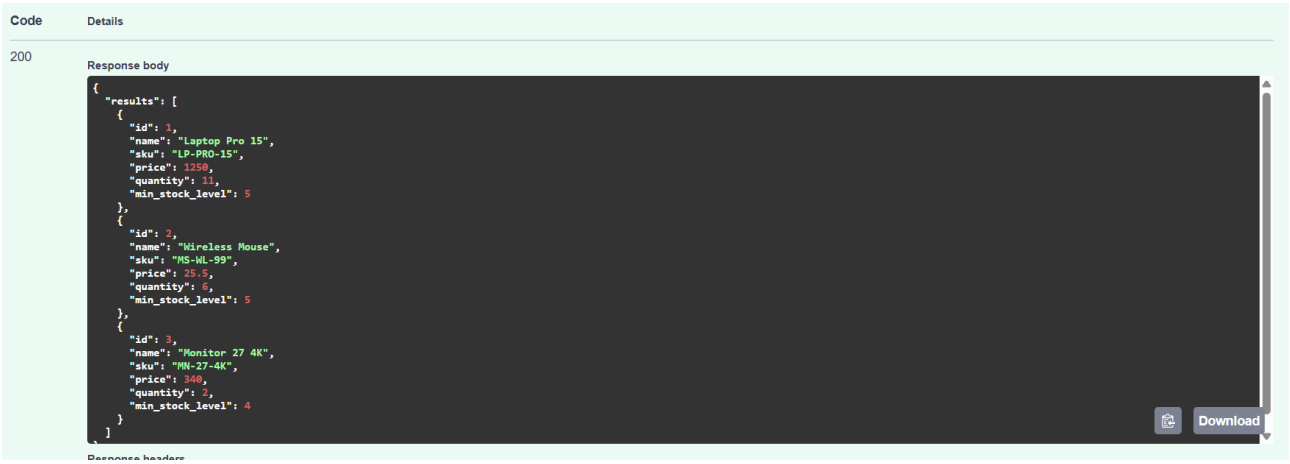


- Переглянула загальну інформацію складу

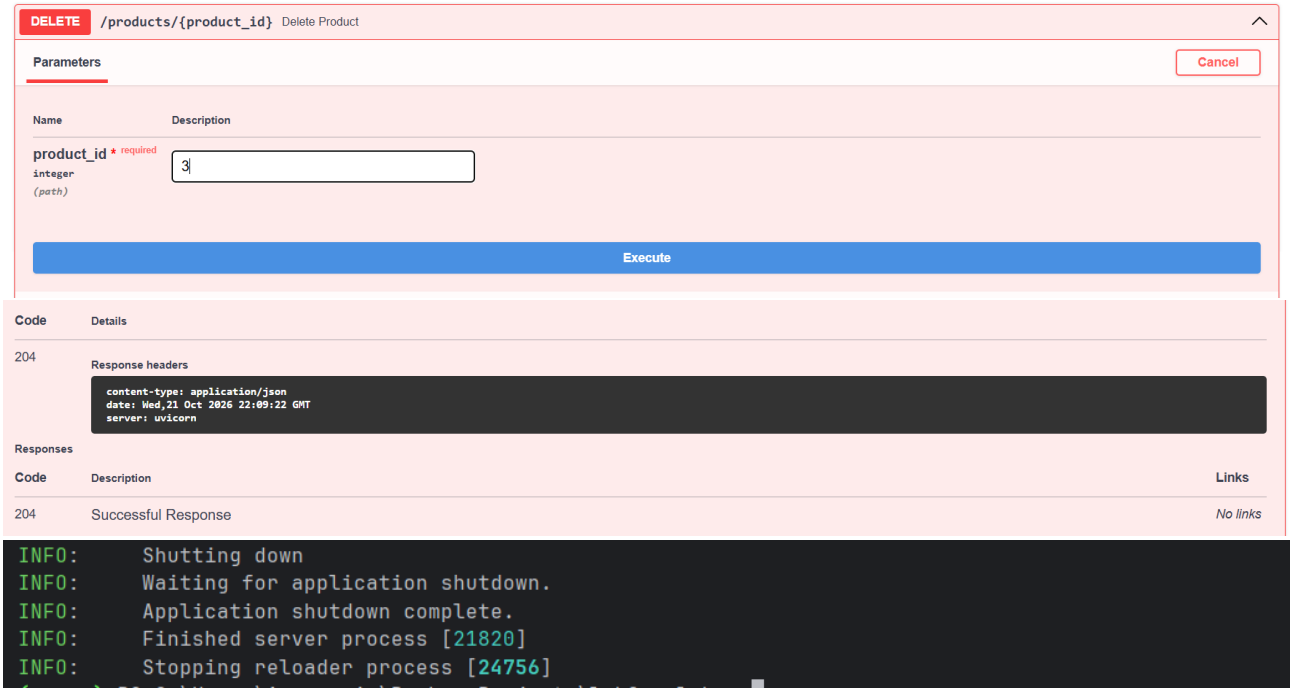


- Перевірила залишки кількох продуктів одночасно





- Видалила продукт



Висновок: під час виконання лабораторної роботи було успішно розроблено REST API для системи обліку складських запасів (Варіант 10) на базі **FastAPI**, **SQLAlchemy** та **Pydantic**. У ході проєкту реалізовано репозиторії та сервіси для управління товарами й складськими операціями, налаштовано асинхронні запити за допомогою **HTTPX**, а також створено автоматизовані інтеграційні тести з використанням **pytest** і бази даних у пам'яті SQLite. Усі розроблені модулі пройшли успішне тестування, що підтверджує правильність бізнес-логіки та готовність системи до використання.